

ЛАБОРАТОРНА РОБОТА № 8

8.1 Тема

Класи та об'єкти: основи об'єктно-орієнтованого програмування в Python.

8.2 Мета роботи

Ознайомитися з основними поняттями об'єктно-орієнтованого програмування (ООП) у мові Python; набути практичних навичок створення класів і об'єктів, використання конструктора класу, атрибутів та методів; навчитися моделювати предметну область із застосуванням класів.

8.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- основні поняття об'єктно-орієнтованого програмування: клас, об'єкт, атрибут, метод, конструктор;
- синтаксис оголошення класів у мові Python;
- призначення та принцип роботи конструктора `__init__`;
- способи доступу до атрибутів і методів об'єктів;
- базові принципи інкапсуляції та використання методів класу.

2) уміти:

- створювати класи та об'єкти в мові Python;
- визначати та використовувати атрибути і методи класу;
- ініціалізувати об'єкти за допомогою конструктора;
- викликати методи об'єктів і працювати з їхніми даними;
- моделювати прості предметні області із застосуванням об'єктно-орієнтованого підходу.

3) набути навичок:

- практичного застосування об'єктно-орієнтованого програмування при розробленні програм;
- структуризації програмного коду з використанням класів;

- аналізу та проектування класів відповідно до поставленого завдання;
- налагодження та тестування програм, що використовують об'єкти і класи.

8.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

8.5 Теоретичні відомості

Поняття об'єктно-орієнтованого програмування

Об'єктно-орієнтоване програмування (ООП) – це підхід до розроблення програмного забезпечення, за якого програма розглядається як сукупність об'єктів, що взаємодіють між собою. Кожен об'єкт поєднує дані та методи їх обробки.

Основними поняттями ООП є:

- клас;
- об'єкт;
- атрибут;
- метод;
- конструктор.

Клас та об'єкт у мові Python

Клас – це шаблон (модель), що визначає структуру та поведінку об'єктів певного типу. Клас описує, які дані зберігає об'єкт та які дії з ними можна виконувати.

Об'єкт – це конкретний екземпляр класу, створений на його основі. Один клас може мати необмежену кількість об'єктів, кожен з яких має власні значення атрибутів.

Оголошення класу в Python здійснюється за допомогою ключового слова `class`.

Атрибути та методи класу

Атрибути класу – це змінні, що описують властивості об'єкта. Атрибути зберігають дані, які характеризують стан об'єкта.

Методи класу – це функції, визначені всередині класу, які описують дії, що може виконувати об'єкт. Методи мають доступ до атрибутів об'єкта.

Для звернення до атрибутів і методів використовується оператор крапки (`.`).

Конструктор класу

Конструктор – це спеціальний метод класу, який автоматично викликається під час створення об'єкта. У Python конструктором є метод `__init__`.

Основне призначення конструктора:

- ініціалізація атрибутів об'єкта;
- задання початкового стану об'єкта.

Метод `__init__` обов'язково має перший параметр ***self***, який є посиланням на поточний об'єкт.

Ключове слово *self*

self – це посилання на об'єкт, для якого викликається метод. Воно дозволяє:

- отримувати доступ до атрибутів об'єкта;
- викликати інші методи цього ж об'єкта.

Використання ***self*** є обов'язковим у визначенні методів класу.

Інкапсуляція в Python (базове уявлення)

Інкапсуляція – це принцип ООП, що полягає в об'єднанні даних та методів їх обробки в одному класі. Вона дозволяє приховати внутрішню реалізацію об'єкта та надавати доступ до даних через методи.

У Python інкапсуляція реалізується на рівні домовленостей (іменування атрибутів і методів), що забезпечує логічну ізоляцію даних.

Створення та використання об'єктів

Створення об'єкта класу здійснюється шляхом виклику імені класу з передаванням необхідних аргументів у конструктор. Після створення об'єкта можна:

- звертатися до його атрибутів;
- викликати методи класу для виконання операцій.

8.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

8.7 Загальне завдання

Написати програму, яка:

- вводить початкові дані;
- формує результат з застосуванням концепції ООП;
- виводить результат роботи.

8.8 Індивідуальні завдання

- 1) Реалізувати клас для студентів, який зберігає ім'я та прізвище.
- 2) Створити клас, що містить оцінки студентів та розраховує середній бал.
- 3) Реалізувати клас для обчислення віку студента на основі року народження.

- 4) Створити клас для студентів з можливістю додавати оцінки та зберігати їх у списку.
- 5) Написати клас для управління бібліотекою (додавання, видалення, перегляд книг).
- 6) Реалізувати клас для моделювання покупки в магазині з обчисленням загальної вартості.
- 7) Створити клас для зберігання особистих даних співробітників підприємства.
- 8) Написати клас, який обчислює площу та периметр прямокутника.
- 9) Реалізувати клас для створення та редагування записів у календарі.
- 10) Створити клас для обчислення середнього значення групи оцінок.

Нульовий варіант (приклад реалізації)

Створити клас `Student`, який зберігає ім'я, прізвище та вік студента. Реалізувати метод для виведення інформації про студента на екран. Створити один об'єкт класу та перевірити роботу програми.

Програмний код

```
# Лабораторна робота №8
# Оголошення класу Student
class Student:
    # Конструктор класу (метод ініціалізації об'єкта)
    def __init__(self, first_name, last_name, age):
        # Атрибути класу
        self.first_name = first_name # ім'я студента
        self.last_name = last_name   # прізвище студента
        self.age = age               # вік студента

    # Метод для виведення інформації про студента
    def show_info(self):
        print("Ім'я:", self.first_name)
        print("Прізвище:", self.last_name)
        print("Вік:", self.age)

# Створення об'єкта класу Student
student1 = Student("Іван", "Петренко", 18)

# Виклик методу класу для виведення інформації
student1.show_info()
```

Пояснення до коду

- ***class Student***: – оголошення класу, який описує студента.
- ***__init__*** – конструктор класу, що автоматично викликається під час створення об'єкта.
- ***self*** – посилання на поточний об'єкт класу.
- ***first_name, last_name, age*** – атрибути класу.
- ***show_info()*** – метод класу для відображення даних студента.
- ***student1 = Student(...)*** – створення об'єкта класу.
- ***student1.show_info()*** – виклик методу об'єкта.

Результат роботи

```

PS E:\GoogleDrive\python-basic> e:: cd 'e:\GoogleDrive\python-basic'; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '5578' '--' 'E:\GoogleDrive\python-basic\test.py'
Ім'я: Іван
Прізвище: Петренко
Вік: 18
PS E:\GoogleDrive\python-basic>
  
```

8.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

8.10 Контрольні питання

- 1) Як визначити клас у мові Python?
- 2) Що таке конструктор класу і для чого він використовується?
- 3) Як створюються об'єкти класу?
- 4) Чим методи класу відрізняються від звичайних функцій?
- 5) Що таке спадкування та як воно реалізується в Python?

8.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>
- 2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>